

LLM_Feedback

PROMPT:

This is my one page writeup! The code is also attached as a file.

Introduction

This project focuses on predicting loan defaults for IncludiFinance. The goal is to identify borrower characteristics that signal higher default risk and recommend a model that balances predictive performance with interpretability. The broader objective is to help the company manage risk effectively while still providing credit to customers who can benefit.

Data & Methodology

We used historical loan data including borrower demographics, credit history, and loan details. Missing values were handled and categorical variables were encoded. Because defaults are much less common than non-defaults, we applied oversampling to balance the dataset. The data was split 75/25 with stratification to preserve class balance. For modeling, we tested Random Forest, XGBoost, a standard neural network (MLP), and a calibrated MLP. Class weights were applied in Random Forest and the `scale_pos_weight` parameter in XGBoost to account for imbalance. Hyperparameters were tuned for each model, including learning rate, maximum depth, and number of trees for XGBoost, number of trees for Random Forest, and 2–3 hidden layers with 64–256 nodes plus early stopping for the MLPs. Model performance was evaluated using F1 score, precision, recall, AUC, a custom financial metric I created to measure the practical impact of predictions, training time, and interpretability through feature importance and SHAP values.

Exploratory analysis revealed some clear patterns in borrower risk. Defaults tend to cluster among people with mid-range credit scores and moderate debt levels. Borrowers with higher credit scores rarely default, even if they carry a lot of debt, though there are fewer examples in that group. The size of the loan itself didn't seem to have a strong effect on defaults.

Results & Analysis

Model performance varied across the board, with XGBoost slightly outperforming the others. Its AUC reached around 0.764, a bit higher than Random Forest at 0.744 and the MLPs at 0.745, meaning XGBoost ranks potential defaults more effectively and gives slightly clearer risk signals across different probability thresholds. Random Forest trained faster than XGBoost and the MLPs, taking only moderate time to fit,

while XGBoost required longer but remained practical. The neural networks were more computationally intensive, especially with multiple hidden layers and early stopping, though the calibrated MLP in particular proved slower to tune and tricky to use operationally.

The threshold analysis highlights the tradeoff between managing risk and maximizing financial impact. At an F1-optimal threshold of 0.65, XGBoost flags roughly 37 percent of defaults while approving 95 percent of applicants, generating a net value around \$153 million. Shifting to the financial-optimal threshold of 0.70, net value rises to about \$202 million, catching closer to 30 percent of defaults and approving 93 percent of applicants. At very high thresholds near 0.85, XGBoost flags only a small fraction of defaults, approves almost everyone, and net value drops to roughly \$97 million. The MLP Raw model reaches a peak net value similar to the calibrated MLP at about \$171 million around a threshold of 0.70 and retains flexibility across the probability range, operating somewhat like XGBoost. Calibrated MLP peaks at the same net value but at a very low threshold around 0.20, and above 0.50 it flags almost no defaults, making it less practical in real-world use. Random Forest generally underperforms in both predictive accuracy and net financial impact, making it the weakest option despite its speed and interpretability.

Model Comparison & Discussion

Choosing the right model depends on InclusiFinance's priorities. XGBoost strikes the strongest balance between predictive performance, financial outcomes, and operational practicality. It captures non-linear relationships, effectively ranks risk, and can be tuned across thresholds to balance default coverage with applicant approvals. MLP Raw is a reasonable backup because it can achieve comparable net value, but it requires more computational resources and is less interpretable. Calibrated MLP, while theoretically flexible, is difficult to use in practice because its probability outputs are skewed, and high thresholds flag very few defaults. Random Forest is fast and interpretable but falls short in both predictive power and net financial value, so it is not recommended for deployment.

Overall, for a threshold around 0.70, XGBoost maximizes net value while still capturing a meaningful portion of defaults, offering a practical way for InclusiFinance to manage risk without being overly restrictive. Threshold choice can be adjusted depending on whether the focus is on approving more applicants or prioritizing profit, but XGBoost provides the clearest path forward.

Conclusion & Future Steps

Deploying XGBoost around a threshold of 0.70 gives InclusiFinance the best balance between flagging potential defaults and approving qualified borrowers. This approach maximizes financial value while still

capturing a meaningful portion of high-risk cases, helping the company manage risk without unnecessarily restricting access to credit. Moving forward, the model should be monitored and recalibrated as borrower behavior evolves, and additional data sources could be incorporated to improve predictions and ensure fairness. The threshold can be adjusted depending on whether the focus is on serving more applicants or prioritizing profit, giving flexibility in operational decision-making. Overall, these insights provide a practical, actionable foundation for more informed and strategic risk management.

Act as a senior data scientist specializing in FinTech and risk modeling. Provide critical feedback on this project. Review my 1-page write-up for clarity, structure, and business impact. Review my Python code for best practices, especially concerning model implementation and handling imbalanced data. Suggest specific improvements for both.

UNEDITED FEEDBACK:

Your one-page write-up effectively summarizes a solid loan default prediction project on what appears to be the Home Credit dataset, highlighting XGBoost's edge in AUC (0.764) and financial impact (\$202M net at threshold 0.70). The code demonstrates strong EDA, feature engineering (e.g., ratios like CREDITINCOMERATIO, aggregates like EXTSOURCEMEAN), and thorough tuning with grid search and CV, though it lacks explicit SHAP despite mentions.^[1]

Write-up Strengths

The structure flows logically from data/methods to results, model comparison, and recommendations, making it concise for stakeholders. Business framing shines through the custom financial metric (LGD 50%, margin 12%), threshold analysis, and InclusiFinance-specific tradeoffs (e.g., approving 93% at 0.70 while catching 30% defaults). Metrics like AUC, F1, precision/recall, and net value provide a balanced view beyond accuracy.^[1]

Write-up Issues

Clarity suffers from vague phrasing like "mid-range credit scores" without specifics (e.g., EXT_SOURCE_2 0.3-0.5) or exact baseline comparisons (e.g., naive model's ~\$0 net). Structure packs too much into "Results & Analysis," blurring performance vs. threshold insights; split into separate sections. Business impact underemphasizes fairness (e.g., no demo/region bias checks) or deployment costs, critical for FinTech regulators. Exploratory findings (e.g., loan size irrelevance) feel anecdotal without visuals or stats like correlation values.^[1]

Aspect	Strength	Weakness
Clarity	Concise metrics, threshold examples	Vague patterns (e.g., "moderate debt")
Structure	Logical progression	Overloaded results section
Impact	Financial metric ties to ROI	Misses fairness, costs

Code Strengths

Best practices abound: stratified splits preserve 8.07% default rate; missing handling (drop >25%, indicators, median impute) is pragmatic; DAYSEMPLOYED sentinel fixed correctly; 30 engineered features (e.g., EXTSOURCEBYAGE interactions) boost signal; imbalance via class_weight='balanced' (RF), scale_pos_weight=11.39 (XGB), class weights (MLP); grid search (lr 0.01-0.2, depth 1-6, n_estimators 50-350) and 5-fold stratified CV validate stability (AUC SD 0.0037).^[1]

Code Issues

Imbalanced data handling mixes methods inconsistently—RF/XGB use weights (good for probabilities), but write-up mentions oversampling without code evidence (risks leakage/overfitting). MLP class weights (0:0.54, 1:6.19) work but raw probs overconfident (mean 0.427 vs. 0.08 base); Platt calibration helps but fitted on val set (leakage). No cross-val for MLP; threshold financial eval uses val AMTCREDIT (fine for demo, but production needs OOS). Missing SHAP/partial dependence despite write-up claims—feature importances (EXTSOURCEMEAN 0.156 avg) are Gini/gain-based, not model-agnostic. No fairness (e.g., demo parity) or leakage checks (e.g., future leaks in Home Credit).^[1]

Model	Imbalance Handling	AUC	Financial Peak
RF	class_weight=balanced	0.744	Lower than XGB
XGB	scale_pos_weight=11.39	0.764	\$202M @0.70
MLP Raw	class weights	0.750	\$171M @0.70
MLP Calib	Platt on raw	0.745	\$171M @0.20

Specific Improvements: Write-up

- **Quantify EDA:** Add "Defaults cluster at EXT_SOURCE_2 0.25-0.5 (corr -0.16) with CREDITINCOMERATIO 2-6."^[1]
- **Restructure:** New headers "## Model Performance," "## Threshold Tradeoffs," "## Recommendation."
- **Enhance Impact:** Include table above; note "XGBoost lifts net value 18% over MLP (\$202M vs. \$171M); monitor fairness via demo AUC gaps."^[1]
- **Future Steps:** Specify A/B test, retrain quarterly, add bureau data.

Specific Improvements: Code

- **Imbalance:** Standardize to SMOTE (pipeline: Pipeline([('smote', SMOTE()), ('xgb', XGBClassifier())])) + CV; drop oversampling or clarify if used.^[1]
- **Interpretability:** Add SHAP: `explainer = shap.TreeExplainer(xgbfinal); shap_values = explainer.shap_values(Xval); shap.summary_plot(shap_values, Xval)`.
- **MLP:** Use `CalibratedClassifierCV(model, method='isotonic', cv=5)` in CV; early stopping via `EarlyStopping(monitor='val_auc')`.

- **Robustness:** Pipeline with ColumnTransformer for preprocessing; fairness: `from aif360 import BinaryLabelDataset; compute disparate_impact()`. OOS test split.
- **Efficiency:** XGB `early_stopping_rounds=50`; RF `warm_start=True` for tuning.

These tweaks elevate it to production-ready: XGBoost deployment via ONNX for speed, with threshold as config var.^[1]

Reflection on feedback

The feedback was super helpful in showing me how to make my write-up clearer and actually useful for someone like a risk manager at IncludiFinance. I took a lot of the suggestions, like being more specific in the exploratory analysis and really explaining the tradeoffs in the threshold and financial metric stuff. I also focused the model comparison on the things that actually matter, like interpretability, computational cost, and how practical it is to use, instead of listing every hyperparameter. Adding details about which models do best at different thresholds and how net value changes made the results feel more tied to real business decisions, not just technical numbers.

There were a few things I didn't change that were mostly coding-related. I didn't fully implement SHAP or fairness checks yet because I felt like this project was meant to show patterns and practical recommendations, not a fully production-ready pipeline. I also didn't switch oversampling to SMOTE because I wanted to stay consistent with what I actually ran. I couldn't include every detail in the write-up either, since we were supposed to keep it brief and I was running out of space. For example, don't think I can break up the Results section fully, and talk about all the background on the financial metric. I don't think I can do that because I need to keep the write-up focused and readable for the audience I want. The feedback really helped me see what's worth highlighting in writing versus code, how to explain tricky technical stuff in plain language, and how to make results actionable without overcomplicating things.